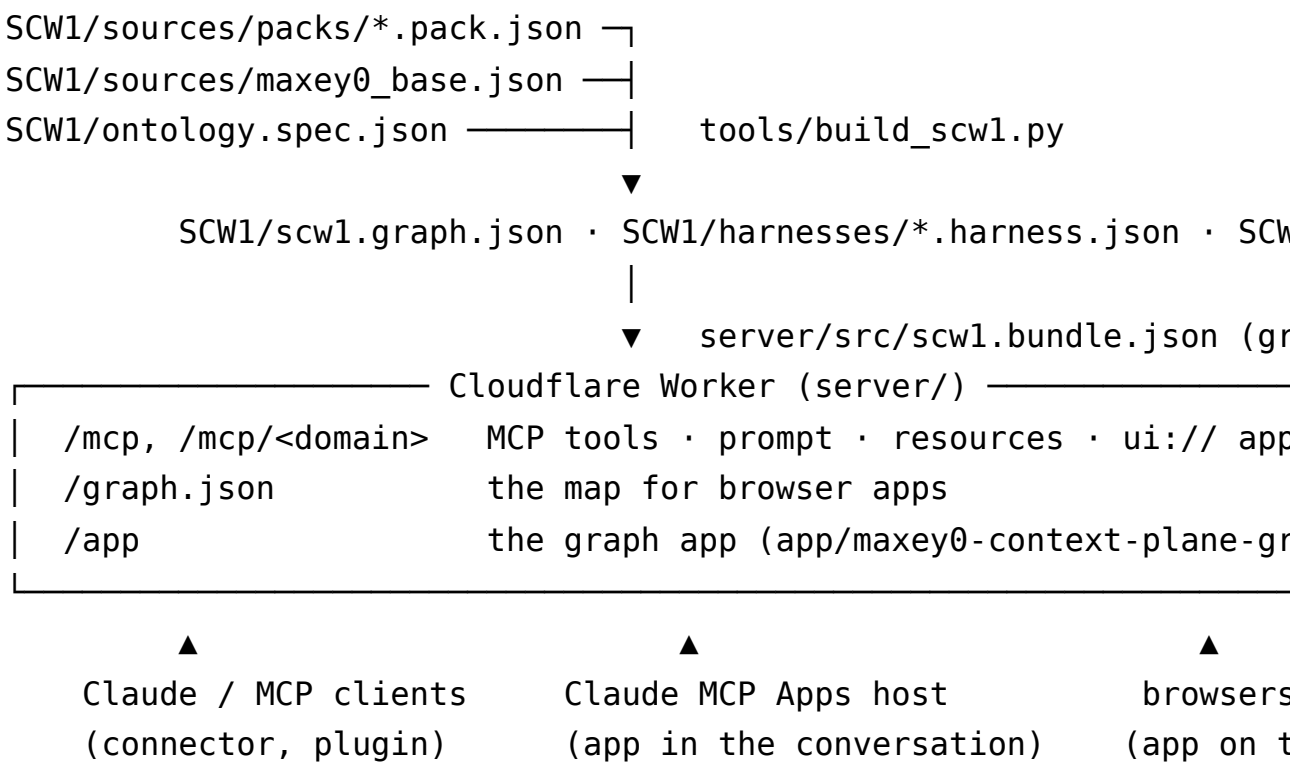
Architecture

One data set, built once, served three ways.



The map

Sources

Each topic starts as a **topic pack** (`human0.topic-pack/1`): a topic, its concepts, skills with their agents and roles, agent names, MCP server rows, a lexicon and provenance. The Maxey0 base graph adds the Maxey0 system's own concepts and agents. Sources are inputs and are never edited by the build.

The ontology spec

`SCW1/ontology.spec.json` holds every structural decision: which domain each topic belongs to, topics and concepts that are merged, moved, renamed or dropped, id

rules, the similarity method and its thresholds, declared links, topic priority, role overrides, and open questions. Changing the map means changing the spec or a pack and rebuilding.

The builder

`tools/build_scw1.py build` applies the spec to the sources and refuses to write anything if a rule breaks. It checks, among others:

- every node has exactly one parent, and every thread reaches the skill level;
- each topic has 3–8 concepts and no concept is named like a topic;
- labels and ids are unique;
- every skill has a primary agent;
- every agent key is a MaxeyN key from `SCW1/maxey_registry.json`, which keeps keys permanent across builds (new agents get the next free number);
- every link meets its threshold.

It writes the graph, one harness per topic, a manifest and build report, and the server bundle. The graph carries a SHA-256 digest; the app records it in saved state.

One similarity method

Search, routing, agent→server links and related concepts all use TF-IDF cosine over one tokenizer (lower case, `[a-z0-9]+` words, stop words and words of two letters or fewer removed, trailing plural *s* folded). Weight = (1 + ln tf) · idf, idf = ln((N + 1)/(df + 1)) + 1, vectors L2-normalised. A pair counts only when it shares at least min(2, terms in the shorter side) words.

Use	Rule
Search and routing	cosine ≥ 0.08
Agent → MCP server	each agent scored by its single best skill; cosine ≥ 0.10; top 2 agents per server
Related concepts	different topics, cosine ≥ 0.40

Use	Rule
Checker inheritance	a checker holds every server its primary holds on the same skill (`via` records the source)

The server's `sim.ts` mirrors the builder's tokenizer and vectoriser exactly; a test checks it reproduces every stored vector.

One embedding field

Every node on all six levels has a vector in the same TF-IDF space (same tokenizer, same idf):

Level	Text it is built from
Domain	label, description, and the text of its topics
Topic	label, description, and its concepts
Concept · Skill	their own text (as used for routing)
Agent	its name and every skill it serves
MCP server	name, description, tool names

Aggregate vectors keep their 48 strongest terms. The builder stacks all vectors into one matrix, takes a deterministic truncated SVD (24 dimensions, ARPACK), normalizes each row, and projects onto the two leading principal axes; the farthest node sits at radius 1. Every level uses the same projection and scale, so a node's location is comparable across levels. `graph.json` carries the locations (`field.xy`); `?index=1` carries the vectors; the `nearest` tool ranks nodes of any level by cosine in that space. The server test checks that topic–concept pairs sit at under half the random-pair distance and that agent–server gates sit closer than random.

Domains and topics, from the concepts up

The domain and topic levels were rebuilt bottom-up (spec `regroup`). Each concept was described by its own text, its skills, the agents that serve them and the servers those agents hold; concepts were grouped on that description into 128 topics of 3–8

concepts, and topics into 11 domains, with placements the data got wrong corrected by hand and marked as such. Concepts, skills, agents, servers and links did not change. Each concept keeps `source_topic`, the pack topic it came from; agent-name uniqueness and checker inheritance are checked against that source topic. A topic pack added later keeps its own topic in the domain named for it in `regroup.pack_domains`.

Observer-lens routing

`route_task` with `mode: "lens"` routes the way an agent inside the graph observes it. The observer starts above the topic plane (or at chosen domains, superposed when more than one), selects up to three topics, and reads the next plane from the selected nodes:

Quantity	Meaning
a = cos(x, p)	how much of candidate x the observer's position p already explains
s = cos(x − a·p, q_p)	what x adds beyond p, against what is left of the task; can be negative
φ_p = λ·a + (1 − λ)·s	the value of x seen from p
ψ(x) = Σ w_p·φ_p, Σ w_p² = 1	the superposed observer; ranked by ψ, selected only when ψ > 0

After each level the task loses the part the selected nodes explain. Each selection records every branch's contribution, and the strongest task matches the lens suppressed are listed with the branches that opposed them. Agents and servers are not chosen by the lens: roles, gates and checker inheritance come from the ontology.

λ was set by calibrating on 400 skill labels with known paths, then testing on 24 tasks not used for calibration:

Router	Skill@1 (labels)	Skill in top 5	Topic@1
flat	0.925	0.998	0.950

Router	Skill@1 (labels)	Skill in top 5	Topic@1
lens, λ = 0.5 / 0.3 (chosen)	0.912	0.970	0.940
lens, λ = 0.65 / 0.45	0.907	0.970	0.935
lens, λ = 0.8 / 0.6	0.902	0.968	0.930
lens, task-driven λ	0.910	0.968	0.940

Labels share the map's own words, so flat ranking is expected to win there; the lens returns at most three skills, which caps top-5. On the 24 held-out tasks the chosen lens matched flat on 21, was better on 2 (a French translation task reached French text translation; a data-pipeline task reached the pipeline builder) and different but not worse on 1. Task-driven λ was worse on 3. Two known flat failures (server logs routed to drone flight logs; the Krebs cycle to the cell cycle) were not fixed by the lens: the wrong topic wins at entry, which is a vocabulary limit, not an observation one. Signed opposition rarely appears among top candidates with word-count vectors, because they carry no negation. Flat stays the default.

Lens through agents and MCP servers, and drift

Below the skill level the lens is structural: agent candidates are only the agents of the chosen skills, and MCP candidates are only the servers gated to the chosen agents. The same ψ chooses among them. λ for agents was calibrated the same way as the upper levels (task = a skill's description; hit = the routed skill's primary agent chosen first; 300 calibration skills, 300 held-out skills):

λ (agent)	calibration	held-out
0.0	0.107	—
0.5	0.453	0.403
0.9 (chosen)	0.570	0.463
1.0	0.580	0.493

λ = 1.0 scores slightly higher but ignores the task residual entirely; 0.9 keeps it. The MCP level has no ground truth and uses the agent value.

Every chosen node records its **drift**: `flat` (cosine with the original task, no observer), `flat_rank` and `lens_rank` among the level's candidates, `drift = ψ − flat` (how much the path above raised or lowered its fit) and `a_obs` (its alignment with the observer). Every level records `task_in`, `task_out` and the share it `explained`. route_task (mode "lens") returns these as SCWs: `SCW-drift:<node>` per chosen node, `SCW-level:<level>` per level, and `SCW-path` around all levels. The values are exact in the TF-IDF space. The app adds where each node sits on its 3D donut seen from above (home) and from the observer (observed), the displacement between them and its length relative to the donut radius. That 3D part is a projection.

Dense second signal (bge-m3)

Every node also has a bge-m3 embedding (Workers AI), centred on the mean node embedding and normalised; the task is embedded per route. Scores live in one inner-product space, = (1 − β)·TF-IDF cosine + β·dense cosine, so flat ranking and the observer lens (a, s, residuals) use both signals with no other change. With β = 0 the space is exactly the TF-IDF space. Calibration on the live server (November 2026):

Flat routing	Skill@1 (300 labels)	Topic@1	Tasks: top skill correct	Tasks: in top 3	Off-map max score
TF-IDF only	0.933	0.950	13/19	14/19	—
β 0.3	0.960	0.970	14/19	16/19	0.412
β 0.5	0.970	0.977	14/19	17/19	0.317
β 0.7 (chosen)	**0.987**	**0.993**	**15/19**	**17/19**	**0.239**
β 0.9	0.980	0.990	13/19	16/19	0.262

Observer lens	Skill@1 (300 labels)	Topic@1	Tasks: top skill correct	Tasks: in top 3
TF-IDF only	0.923	0.943	16/19	16/19
β 0.3	0.960	0.970	16/19	17/19

Observer lens	Skill@1 (300 labels)	Topic@1	Tasks: top skill correct	Tasks: in top 3
β 0.5	0.973	0.980	16/19	17/19
β 0.7 (chosen)	**0.983**	**0.990**	**16/19**	**17/19**
β 0.9	0.980	0.987	14/19	17/19

The lens with dense puts the server-log anomaly spotter first (flat does not), but takes the hike finder over the itinerary planner for "plan a three day hike with campsites", which the fixed judgement counts as wrong. The lens has no match threshold: it selects whenever ψ > 0, so an off-map request still gets a route (highest first-level ψ on the 10 off-map requests: 0.184 at β 0.7).

Tasks are the 20 author-written tasks of the paper's Section 9.2 with the correct skills fixed before any dense run (19 have a correct skill; for "value a startup for a seed round" the map has none). Off-map: 10 author-written requests nothing in the map does ("tell me a joke about penguins"). The threshold 0.25 sits between the off-map maximum (0.239) and the lowest correct top score (0.300) at β 0.7; it was chosen on these same 30 requests, so it is fitted, not tested. What dense fixed: the French translation and hallucination-tracing tasks now reach their skills, and DNS/CDN and server-log anomalies have the right skill second. What it did not fix: the Krebs cycle still reaches the cell cycle, and server logs still put drone flight logs first.

The server

`server/src/` — TypeScript on Cloudflare Workers, no runtime dependencies.

File	Role
`index.ts`	HTTP routing, MCP JSON-RPC (2026-07-28 stateless and the earlier `initialize`), header validation, tools, resources, prompt, `/graph.json`, `/app`

File	Role
`graph.ts`	The map in memory, scoped per domain endpoint; every tool's logic
`sim.ts`	Tokenizer, vectors, cosine with the coverage rule
`generator.ts`	The `topic-graph-generator` prompt and `validate_topic_pack`
`app.ts`	The graph app as an MCP App resource and as the `/app` page

The bundle is imported at build time, so the server has no storage and no network calls. A domain endpoint serves a copy of the map trimmed to that domain.

The app

`app/maxey0-context-plane-graph.html` is one self-contained page (HTML, CSS, JavaScript; no libraries, no external requests other than the map).

It is produced by `app/build_ui.py` from a fixed source page (`app/source/`). Each change is a named replacement at an anchor that must occur exactly once, so the build either applies every change or stops. On load the page fetches `/graph.json?index=1` from the server it came from, converts it to its internal shapes, and uses the server's gates and TF-IDF index, so the page and `route_task` give the same answers.

Rendering is a 2D canvas with its own 3D projection: one 3×3 orientation matrix maps world coordinates to the view, so rotation has no limits. Levels are rings on one vertical axis; a spread path is laid out in world coordinates on each ring; a dragged node's offset is solved in its level's plane and applied to everything downstream of it on the path.

Inside an MCP Apps host the page talks to the host over `postMessage` (Part *The MCP App* in MCP_SERVER.md); in a browser that code stays idle.

The plugin

`plugin/maxey0-context-plane-graph/` contains `.claude-plugin/plugin.json`, `.mcp.json` (the remote server, `type: http`) and two skills in Markdown. It runs no code locally.

Tests

Suite	Covers
`server/test/mcp.test.mjs`	Protocol (both eras), every tool, domain endpoints, resources, prompt, validator, MCP App resource, `/app`
`server/test/ontology.test.mjs`	The built map against the spec: structure, keys, thresholds, checker rule, decisions
`server/test/sim.test.mjs`	`sim.ts` reproduces every stored vector
`app/test_ui.cjs`	The page in Chromium against a local server: load, counts, gates equal the server's, routing equals `route_task`, isolation, spread, drag, rotation, topics, state
`app/test_host.cjs`	The page inside a simulated MCP Apps host: handshake, sizing, tool input, full screen, messages